Практическое занятие № 7

Задачи работы

1. Создать учебную базу данных и таблицу пользователей.
2. Рассмотреть пример небезопасного формирования SQL-запроса.
3. Объяснить, почему конкатенация пользовательского ввода со строкой SQL создаёт уязвимость.
4. Реализовать безопасный запрос с использованием параметров.
5. Проверить обработку некорректного пользовательского ввода.
6. Определить дополнительные меры защиты базы данных от SQL-инъекций.

3. Теоретические сведения

SQL-инъекция — это уязвимость, при которой пользовательский ввод интерпретируется как часть SQL-команды и может изменить её первоначальную логику.

Например, приложение формирует запрос путём объединения SQL-команды и введённого пользователем имени:

```
username = input("Введите имя пользователя: ")

query = (
    "SELECT id, username FROM app_users "
    + "WHERE username = '" + username + "'"
)
```

Проблема заключается в том, что введённые данные становятся частью текста SQL-запроса.

Основной способ защиты — **параметризованные запросы**. При таком подходе SQL-команда и пользовательские значения передаются отдельно.

```
username = input("Введите имя пользователя: ")

cursor.execute(
    "SELECT id, username FROM app_users WHERE username
= %s",
    (username,)
)
```

В этом примере `%s` — обозначение параметра в библиотеке psycopg. Это не обычная подстановка строки: библиотека передаёт значение отдельно от структуры SQL-запроса.

Дополнительные меры защиты:

- минимальные права пользователя приложения;
- проверка формата вводимых данных;
- безопасная обработка ошибок;
- хранение секретов вне исходного кода;
- тестирование запросов на некорректном вводе.

Ход работы

Этап 1. Создание учебной базы данных

Подключитесь к PostgreSQL через pgAdmin или другой SQL-клиент.

Создайте таблицу пользователей:

```
CREATE TABLE app_users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    full_name VARCHAR(100) NOT NULL
);
```

Добавьте тестовые данные:

```
INSERT INTO app_users (username, full_name)
VALUES
    ('student01', 'Иван Петров'),
    ('student02', 'Анна Сидорова'),
    ('teacher01', 'Мария Иванова');
```

Выполните запрос:

```
SELECT id, username, full_name
FROM app_users;
```

Задание: убедитесь, что таблица содержит три записи. Зафиксируйте результат.

Этап 2. Анализ небезопасного запроса

Изучите следующий учебный фрагмент Python:

```
username = input("Введите логин: ")
```

```
query = (
    "SELECT id, username FROM app_users "
    + "WHERE username = '" + username + "'"
)

print(query)
```

В этой части работы необходимо только сформировать и вывести текст запроса. Не подключайте данный фрагмент к реальной базе данных и не выполняйте его с произвольным вводом.

Задание:

1. Объясните, почему объединение SQL-команды и пользовательского ввода опасно.
2. Укажите, какая часть запроса должна оставаться неизменной.
3. Объясните, почему простая проверка длины логина не устраняет проблему SQL-инъекции.
4. Предложите безопасный способ формирования запроса.

Запишите ответы в отчёт.

Этап 3. Реализация безопасного запроса

Установите библиотеку psycopg для Python, если она ещё не установлена:

```
pip install "psycopg[binary]"
```

Создайте файл `safe_query.py`:

```
import os
import psycopg

username = input("Введите логин: ")

connection_string = os.environ["DATABASE_URL"]

with psycopg.connect(connection_string) as conn:
    with conn.cursor() as cursor:
        cursor.execute(
            """
            SELECT id, username, full_name
            FROM app_users
            WHERE username = %s
            """,
```

```
        (username,)
    )

    rows = cursor.fetchall()

    if rows:
        for row in rows:
            print(row)
    else:
        print("Пользователь не найден")
```

Переменная окружения `DATABASE_URL` должна содержать параметры подключения к вашей учебной базе. Установите её в соответствии с используемой операционной системой и не размещайте пароль от БД в исходном коде.

Задание:

1. Запустите программу с логином `student01`.
2. Повторите проверку с логином `teacher01`.
3. Введите несуществующий логин.
4. Введите строку с апострофом, например `student'01`.
5. Убедитесь, что некорректное значение обрабатывается как обычное значение параметра, а не как часть SQL-команды.

Запишите фактические результаты проверки.

Этап 4. Проверка входных данных

Параметризация защищает структуру SQL-запроса, но приложение также должно проверять допустимый формат логина.

Добавьте перед выполнением запроса проверку:

```
import re

if not re.fullmatch(r"[A-Za-z0-9_]{3,30}", username):
    print("Ошибка: логин должен содержать 3–30
символов: "
          "латинские буквы, цифры или знак
подчёркивания.")
else:
    # Выполните параметризованный запрос
    pass
```

Вместо `pass` разместите код выполнения безопасного запроса из предыдущего этапа.

Задание: проверьте следующие значения:

Ввод	Ожидаемый результат
`student01`	Формат допустим; выполняется поиск
`teacher01`	Формат допустим; выполняется поиск
`ab`	Отклоняется из-за длины
`user name`	Отклоняется из-за пробела
`student'01`	Отклоняется проверкой формата

Важно: проверка формата является дополнительной мерой. Основная защита от SQL-инъекции — параметризованный запрос.

Этап 5. Анализ дополнительных мер защиты

Предположим, что приложение университета использует базу данных со сведениями о студентах, преподавателях и оценках.

Определите, какие меры необходимо применить, чтобы снизить риск SQL-инъекций и ограничить последствия возможной ошибки в приложении.

Заполните таблицу:

Проблема	Предлагаемая мера защиты
SQL формируется конкатенацией строк	
Приложение подключается с правами администратора	
Пароль БД записан непосредственно в коде	
Пользователь видит подробную ошибку SQL	
Ввод не проверяется на соответствие формату	

При выборе мер учитывайте принцип наименьших привилегий: приложение должно иметь только те права, которые необходимы для выполнения его функций.